Kunming AI+

An Applied Introduction to AI with PyTorch

Luca Ghafourpour

UNIVERSITY OF CAMBRIDGE
Department of Engineering

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance + *Weights and Biases*
Practical Workshop 3	Initialisation and regularisation
Workshop 4	Convolutional neural networks
Practical Workshop 4	Dimensionality reduction, k-fold CV, confusion matrices
Academic Lecture 1	An introduction to scientific machine learning
Academic Lecture 2	**Advanced methods in scientific machine learning**
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

A recap

1

Scientific laws

Differential equations encode what we know about a system.

2

Numerical simulation

Discretise on a mesh and solve.

3

Scientific ML

Neural networks combine data and physics (e.g. PINNs).

4

This lecture

Learn reusable models across whole families of problems.

A PINN learns the solution to one problem

We showed how to train a neural network that satisfies a specific PDE, with specific data, on a specific domain:

$$\mathcal{D}[\,u(t,x)\,] = f(t,x)$$

$$\mathcal{B}_k[\,u(t,x)\,] = g_k(t,x)$$

We train a network NN(t, x; θ) to approximate the true solution of the PDE u(t, x) with consistent boundary conditions using the combined loss function:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{physics}}(\theta)$$

PINNs require retraining for any condition change

Every new initial condition, boundary condition, or coefficient defines a new problem, and requires a new training run.

❌ **A PINN does not generalise across problems.**

We are right back to the multi-query problem from Academic Lecture 1.

What if we learned the solution operator itself?

PART 1

Neural Operators

From learning functions to learning operators

Standard neural network

A finite-dimensional map between vectors.

$$f_\theta : \mathbb{R}^m \to \mathbb{R}^n$$

Example:
image → label

Neural operator

A map between spaces of functions.

$$\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}$$

Example:
whole input field → whole output field

The PDE solution operator

For a family of problems, the governing equations induce a map from problem specification to solution.

$$\mathcal{G}^{\dagger} : a(x) \longmapsto u(x)$$

Input function $\mathscr{A}$

coefficients, forcing, initial or boundary data

→

Solution operator

the map induced by the equations and constraints

→

Output function $\mathscr{U}$

the solution field, trajectory or response

Operator learning

Idea: Define a finite-dimensional parametric family of functions called neural operators

$$\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}, \quad \theta \in \mathbb{R}^p$$

Goal: learn parameters θ* such that G_{θ^*} approximates $G^\dagger$.

Training neural operators

1. Generate (or measure) many input-solution pairs once

$$\{(a_j,\, u_j)\}_{j=1}^{N}, \qquad u_j = G^{\dagger}(a_j) + \text{noise}$$

2. Fit the neural operator by minimising the error over the output field:

$$\min_{\theta} \quad \mathbb{E}_{a\sim\mu} \left\| G_\theta(a) - G^{\dagger}(a) \right\|_{\mathcal{U}}^{2}$$

The benefits of neural operators over neural networks

Discretisation-invariant

Can be trained on one mesh, but queried at any resolution. The model learns the operator, not a fixed grid.

One model that generalises across inputs

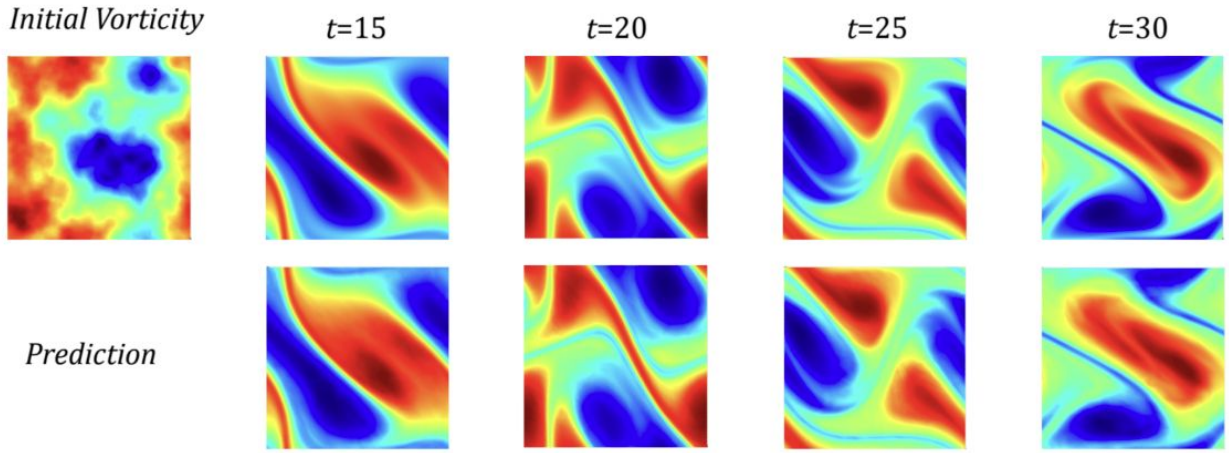
Amortises the cost of solving over thousands of related problems. No retraining needed. **Scalable multi-querying is now possible!**

Mesh-free at inference

Can evaluate the solution at any point y in the domain, not only at training nodes.

Zero-shot super-resolution

An FNO is trained on a 64x64 spatial grid, and recovers the large-scale solution structure when queried at a 256x256 spatial grid, without retraining.



⚠️ **Resolution transfer is not a guarantee that unresolved fine-scale physics will be reconstructed correctly.**

Building a neural operator layer

Standard FNN layer: hidden representation v_t is a vector in $\mathbb{R}^d$

$$\mathbf{v}_{t+1} = \sigma\big(W_t\, \mathbf{v}_t + b_t \big)$$

Neural operator layer: hidden representation v_t is a function $v_t : D \to \mathbb{R}^d$

$$v_{t+1}(x) = \sigma\Big(\underbrace{W_t\, v_t(x)}_{\text{local (pointwise)}} + \underbrace{(\mathcal{K}_\theta v_t)(x)}_{\text{global (nonlocal)}} + \underbrace{b_t(x)}_{\text{bias}} \Big)$$

The global term: a learnable integral kernel

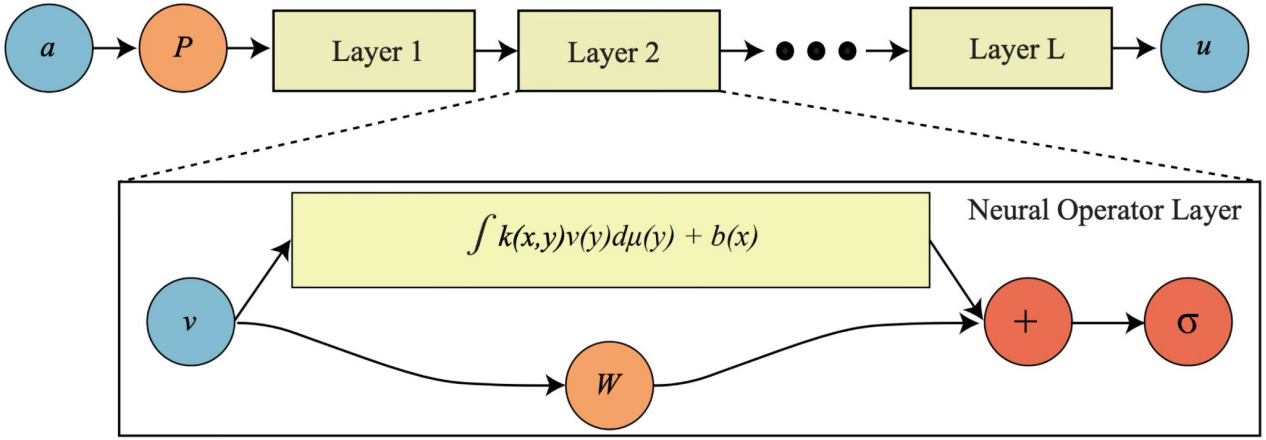
The new non-local term integrates the input hidden representation against a trainable kernel κ_θ.

$$(\mathcal{K}_\theta v_t)(x) = \int_D \kappa_\theta(x, y)\, v_t(y)\, d\nu(y), \qquad x \in D$$

$$\kappa_\theta : D \times D \to \mathbb{R}^{d_{t+1} \times d_t}$$

Integrating over all $y \in D$ allows for all points in the domain to interact.

The Neural Operator Architecture



1. **Lifting** $v_0 = P_\theta(a), \qquad v_0 : D \to \mathbb{R}^{d_0}$

2. **Operator layers** $v_{t+1} = \Phi_t(v_t), \qquad t = 0, \ldots, T-1$

3. **Projection** $u = G_\theta(a) = Q_\theta(v_T) \in \mathcal{U}$

From a general kernel to a convolution

The integral against a general kernel is expensive. Let's make a design choice:

$$\kappa_\theta(x, y) := \kappa_\theta(x - y)$$

As a result, the non-local term becomes a convolution:

$$(\mathcal{K}_\theta v_t)(x) = \int_D \kappa_\theta(x - y)\, v_t(y)\, d\nu(y) = (\kappa_\theta * v_t)(x)$$

By the convolution theorem, a convolution is a pointwise product in Fourier space:

$$(\mathcal{K}_\theta v_t)(x) = \mathcal{F}^{-1}\Big(\, \mathcal{F}(\kappa_\theta) \cdot \mathcal{F}(v_t) \,\Big)(x)$$

Fourier Kernel

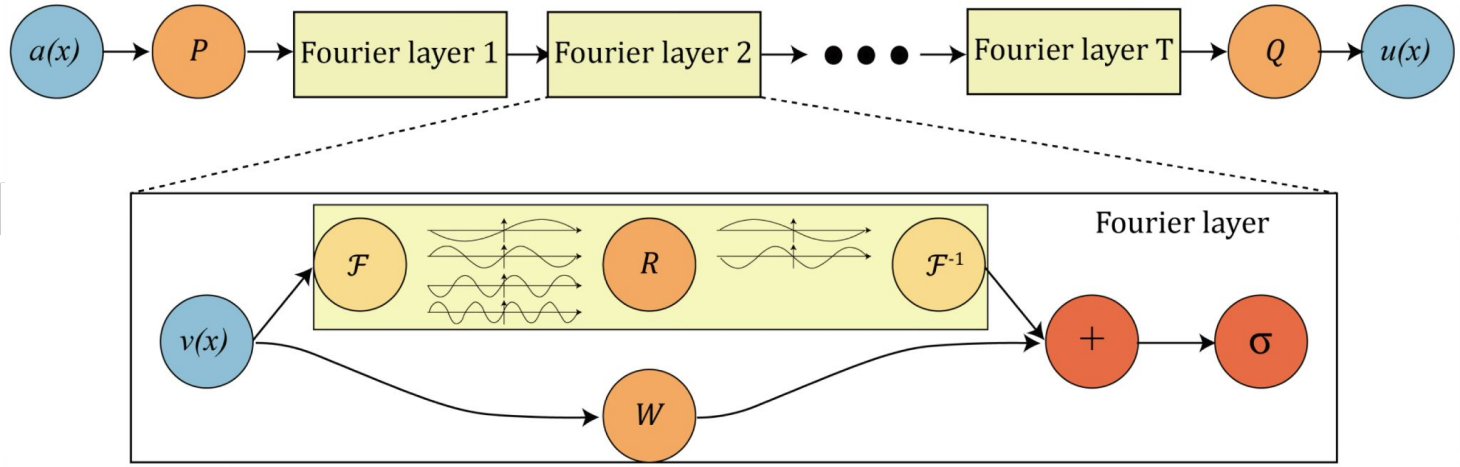
Approach: learn the multiplier R_θ (the transform of the kernel) instead of the kernel itself:

$$(\mathcal{K}_\theta v_t)(x) := \mathcal{F}^{-1}\Big(R_\theta \cdot \mathcal{F}(v_t) \Big)(x),$$

$$R_\theta = \mathcal{F}\left(\kappa_\theta\right) \in \mathbb{C}^{k_{\max} \times d \times d}$$

We define a maximum number of frequencies, k_{max}, that we wish our multiplier to keep. All frequencies above this max are set to 0.

The Fourier Neural Operator layer

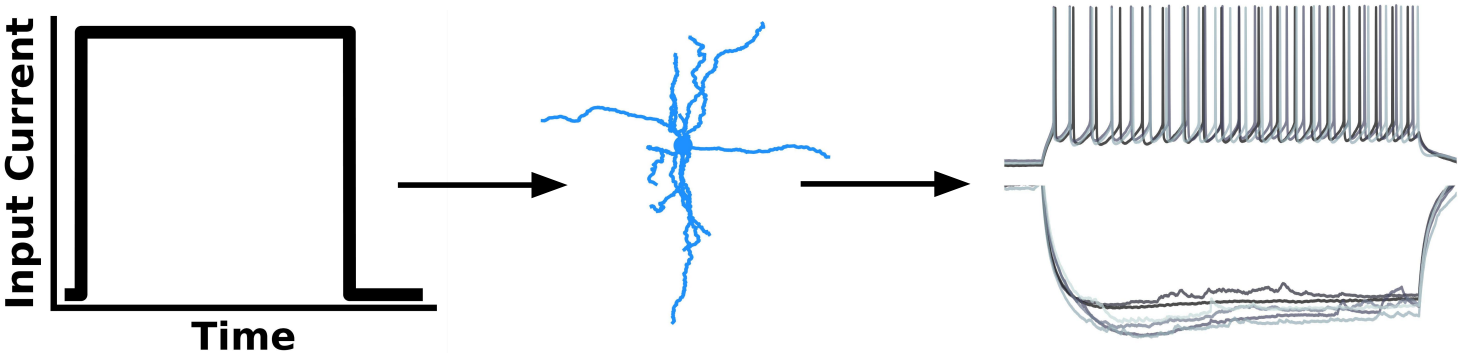


$$v_{t+1}(x) = \sigma\Big(W_t\, v_t(x) + \mathcal{F}^{-1}\big(R_\theta \cdot \mathcal{F}(v_t)\big)(x) \Big)$$

FNOs for Neuroscience

Modelling real neurons

Real neurons exhibit trial-to-trial variability:



Question: Can we simulate neurons with trial-to-trial variability?

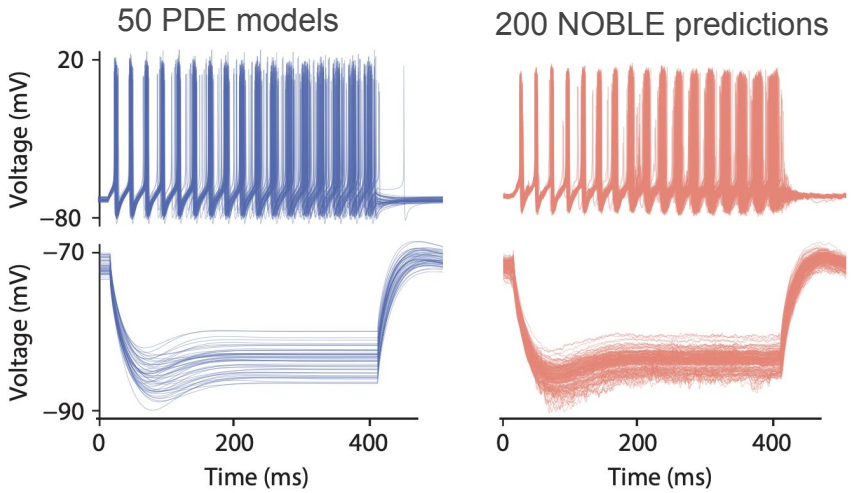
Biorealistic Neuron Modelling

- A single neuron's electrical behaviour can be described using coupled differential equations that, for a given electrical input, return the neuron’s voltage response.
- Such equations are deterministic: for a given input, we get one output.
- Real neurons though are not deterministic! Instead, people use many sets of differential equations, with slightly different coefficients, to model the same neuron.

NOBLE

NOBLE uses an FNO to learn a surrogate across a family of biophysical neuron models, mapping biologically informed neuron representations and stimuli to voltage responses.

It interpolates between modelled neurons and predict voltage responses for previously unseen representations within the learned distribution.



39th Conference on Neural Information Processing Systems (NeurIPS 2025).

Summary: Neural operators vs classical solvers

Property	Classical solver	Learned operator
Cost per new query	hours on a cluster	one forward pass (ms–s)
Reusable across problems	no; re-solve each time	yes; one trained model
Resolution	fixed by the mesh	query at any resolution

PART 2

Generative Models

Generative model motivation

Consider we have a collection of samples $x_1, x_2, \ldots, x_n$ that are drawn from some complex data generating distribution p_{data} that we do not have access to explicitly.

Examples of p_{data} :

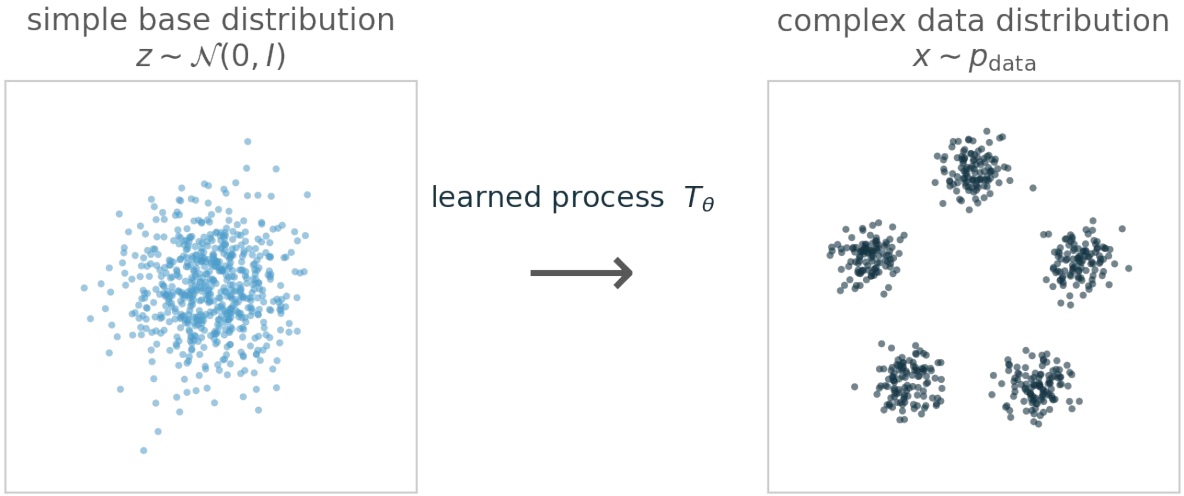
- Weather forecast simulations
- Natural images of cats
- 3D protein structure

Task: Generate new synthetic samples such that they look like they were drawn from p_{data}

💡 p_{data} is intractable. Could we instead sample from a simpler, tractable distribution and learn a transformation that maps those samples to ones that resemble draws from p_{data}?

Generative model setting

$$z \sim p_{\text{simple}} \xrightarrow{T_\theta} x = T_\theta(z) \sim p_{\text{data}}$$



Two methods are most prevalent

Diffusion

Slowly add noise to data, then learn to reverse the noise back to the data. A network denoises step by step, from pure noise back to a sample.

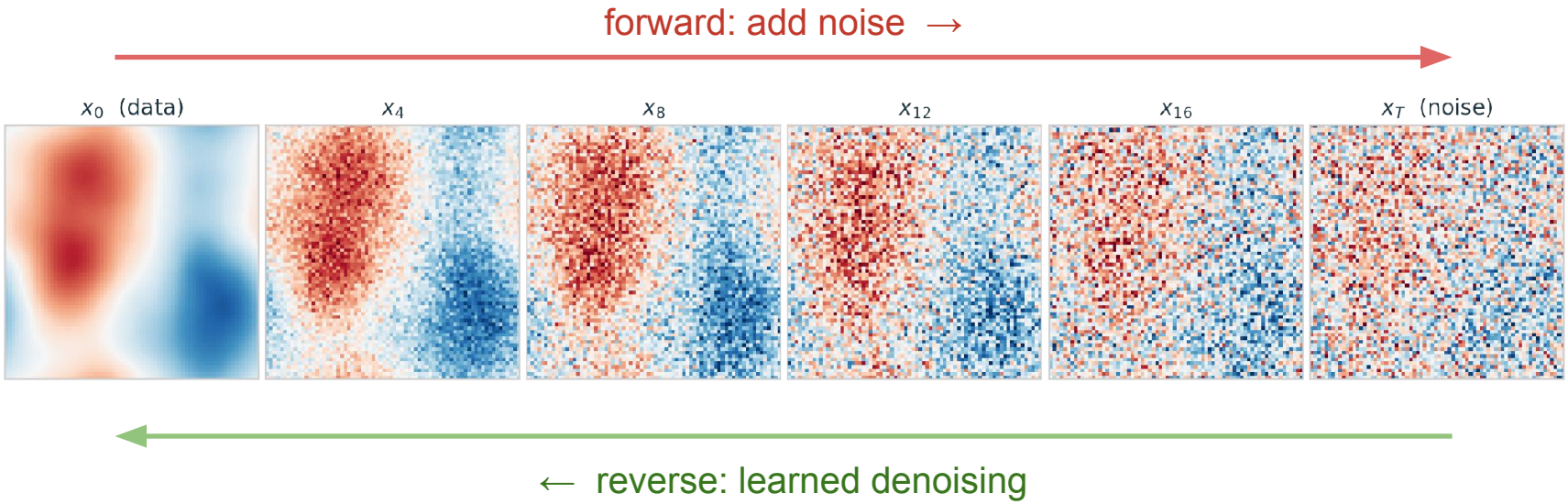
Flow matching

Define a direct path from noise to data, then learn the velocity of a vector field that carries samples along it. Integrate an ODE along the path generates new samples.

Diffusion-Based Generative Modelling

Take a true sample from p_{data} and gradually add Gaussian noise until the data becomes pure noise.

Train a network to reverse the noising one step at a time.



Denoising Diffusion Probabilistic Model (DDPM)

Forward: add a little Gaussian noise each step on a fixed schedule $\beta_1,\ldots,\beta_T$:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t;\ \sqrt{1-\beta_t}\,x_{t-1},\ \ \beta_t \mathbf{I}\big)$$

Conveniently, we can jump straight to any step t from the clean data x_0 in closed form:

$$x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,\mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$$

Reverse: A single network, ε_θ takes as input a noised sample x□ and time step t, and predicts how much noise was added to it. A simple regression task!

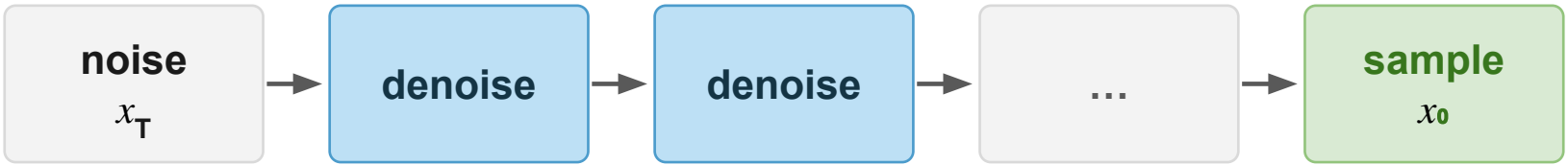
$$\mathcal{L}(\theta) = \mathbb{E}_{x_0,\varepsilon,t}\Big[\ \big\|\,\varepsilon - \varepsilon_\theta(x_t,t)\,\big\|^2\ \Big]$$

Sampling from DDPM

To sample from our approximation to p_{data}, first sample x_T from a standard Gaussian. Then, apply the denoiser step by step back to t = 0:

$$x_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right) + \sigma_t z$$

where z ~ N(0,I).

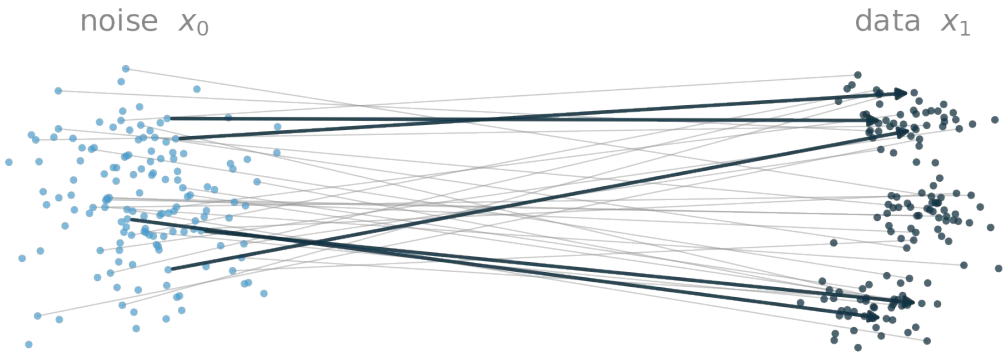


Diffusion samples of synthetic faces of people who don't exist

Flow-Based Generative Modelling

Pick a simple path that transports a noise sample into a data sample (e.g. a straight line between them).

Train a network to predict the velocity that moves you along that path.



Flow matching

Let's take the simplest interpolant, a straight line:

$$x_t = (1-t)\,x_0 + t\,x_1, \qquad x_0 \sim \mathcal{N}(0, I), \quad x_1 \sim p_{\text{data}}$$

Idea: Learn a velocity field v_θ that defines how a given sample should move at each point and time.

$$\frac{dx_t}{dt} = v_\theta(x_t, t), \qquad t : 0 \to 1$$

Generate samples from the target distribution by starting at noise and integrating the field from t = 0 to 1:

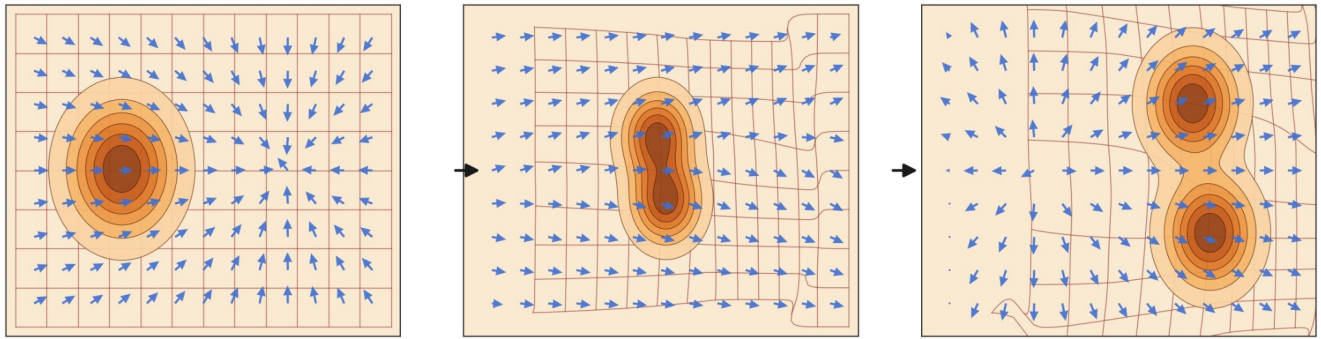
$$x_1 = x_0 + \int_0^1 v_\theta(x_t, t)\,dt$$

Training Flow matching

For the straight line interpolation path we have that $\frac{dx_t}{dt} = x_1 - x_0$

Hence, similar to DDPM, we can train a network v_θ that learns this velocity field by regression!

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\, x_0,\, x_1} \left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2$$



How can we use these generative models for answering questions important to scientists?

Conditioning

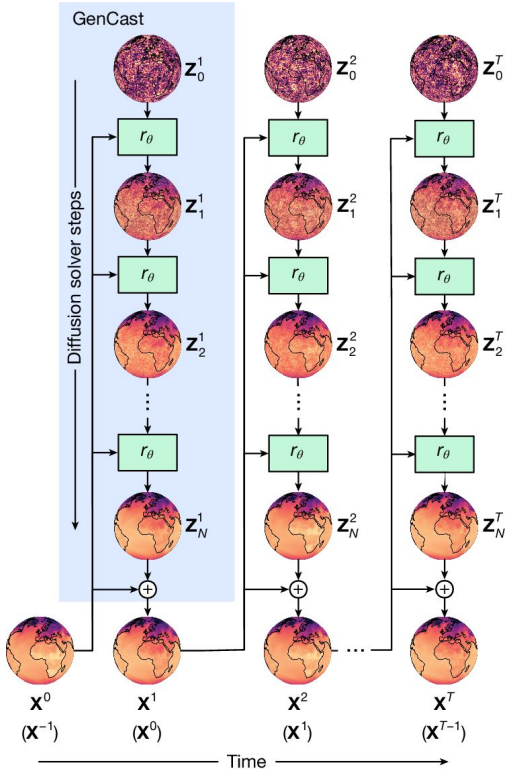
Both flow and diffusion based methods can be conditioned by some condition c to sample from p(x | c) by feeding it as an additional input.

$$\varepsilon_\theta(x_t, t, c) \qquad \text{or} \qquad v_\theta(x_t, t, c)$$

Conditioning teaches the network to turn noise into a sample that satisfies c, so generation becomes targeted and not random.

Domain	Conditioning c	Generates x ~ p(x | c)
Weather	the current atmospheric state	an ensemble of plausible next states
Proteins	an amino-acid sequence	the folded 3-D structure
Materials	a target property (e.g. band gap)	a crystal that has it

GenCast: a conditional diffusion model for weather



Diffusion rollout over time

GenCast rolls the weather forward in time. Each forward 12 hour forecast is sampled from a noise sample using a diffusion model conditioned on the two previous weather states:

$$\mathbf{X}^t \sim p_\theta\left(\mathbf{X}^t \,\middle|\, \mathbf{X}^{t-1}, \mathbf{X}^{t-2}\right)$$

To sample the next forecast, start from noise and run N diffusion solver steps of the denoiser r_θ, then add the result to the current state:

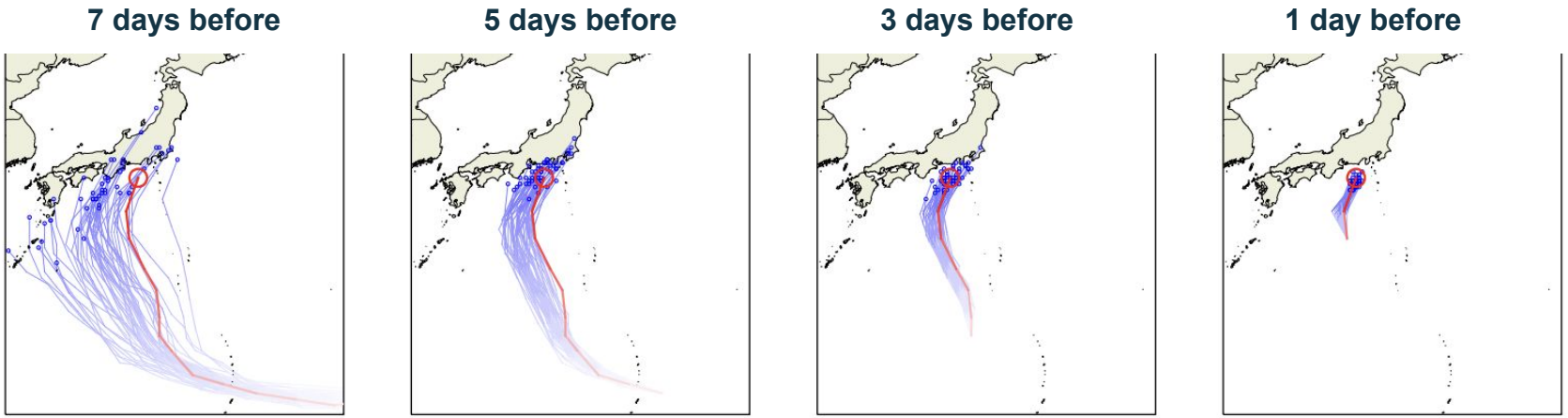
$$\mathbf{z}_0 \sim \mathcal{N}(0, I) \xrightarrow{N \times r_\theta} \mathbf{z}_N, \qquad \mathbf{X}^t = \mathbf{X}^{t-1} + \mathbf{z}_N$$

A full 15-day global forecast takes ~8 minutes on a single TPU.

Enables generation of ensembles of weather forecasts and uncertainty quantification.

SUCCESS STORY IN WEATHER

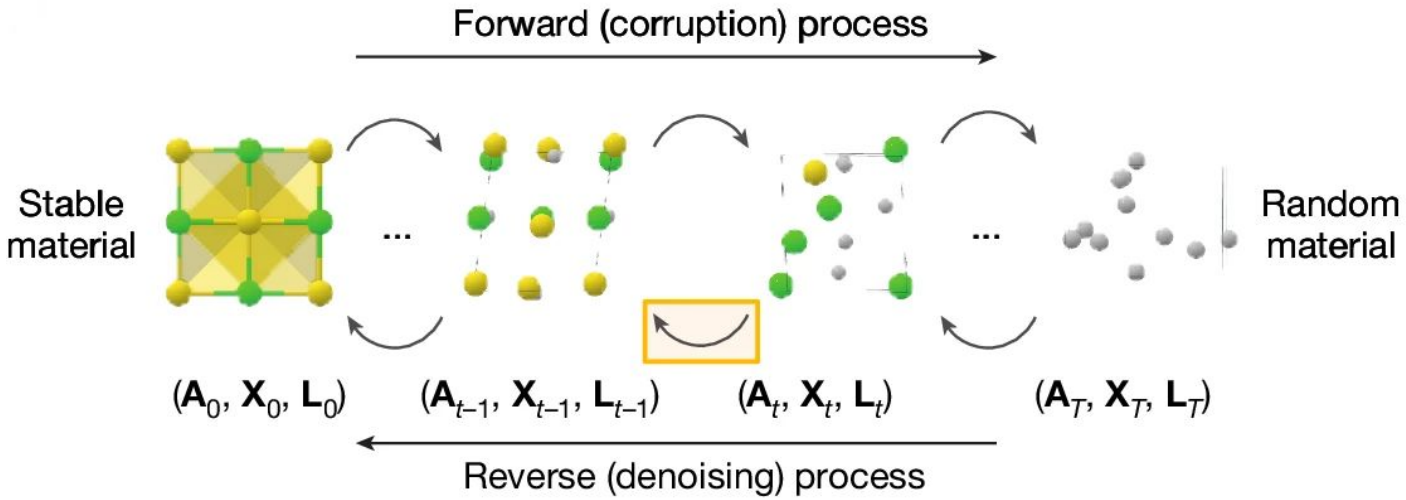
Forecasting where a typhoon will go



- Each ensemble member is one sampled trajectory. Together they map the plausible storm tracks (blue) around the true trajectory (red).
- The ensemble spread represents the model's forecast uncertainty

GenCast beats the leading operational ensemble on ~97% of targets, in minutes rather than hours.

MatterGen: a diffusion model over crystals



A material **M** is the collection of its atom types, coordinates, and lattice:

$$\mathbf{M} = \Big(\underbrace{A}_{\text{atom types}} , \underbrace{X}_{\text{coordinates}} , \underbrace{L}_{\text{lattice}} \Big)$$

SUCCESS STORY IN MATERIALS

Designing materials backwards

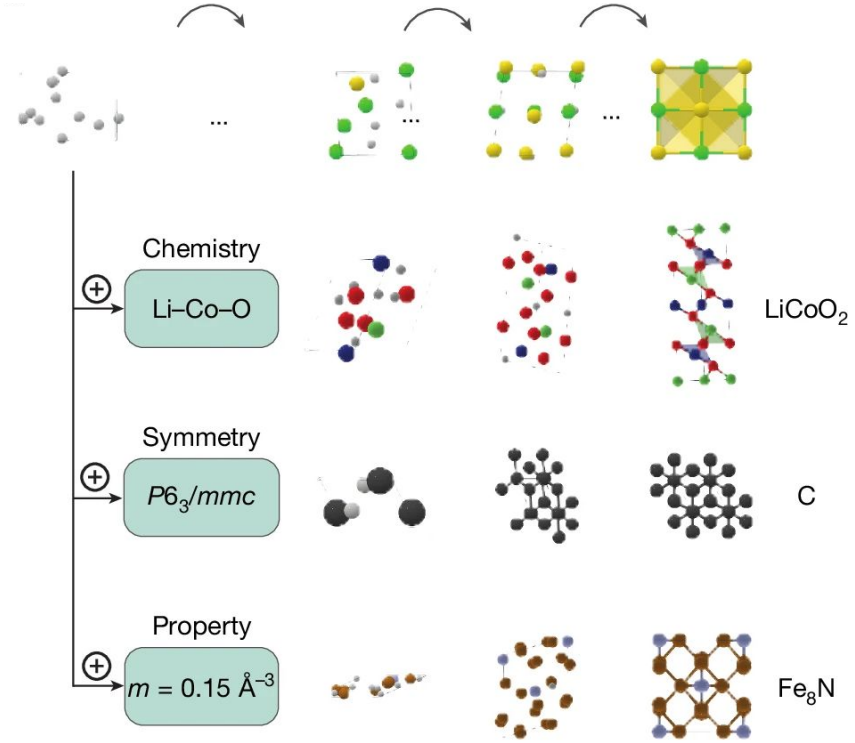
Specify a condition c (a chemistry, a symmetry, or a target property) and MatterGen proposes candidate crystals predicted to satisfy it.

A candidate crystal is sampled from noise using a diffusion model conditioned on the desired condition:

$$\mathbf{M} \sim p_\theta(\mathbf{M} \mid c)$$

> 2x as likely to be novel and stable as prior generative models.

~10x closer to the local energy minimum.



PART 3

The Future?

🤔 **Today's models are mostly trained per problem. Can we have a problem-agnostic model?**

💡 **Pre-train one large model on broad scientific data, then adapt it cheaply to many tasks.**

Foundation Models in Science

1. Pre-training

One large model trains on a huge, diverse dataset. The aim is to build general-purpose representations of the data.

2. Transfer learning

The learnt representations capture structure that is shared across problems. What the model learned carries over to tasks it never saw during training.

3. Fine-tuning on new task

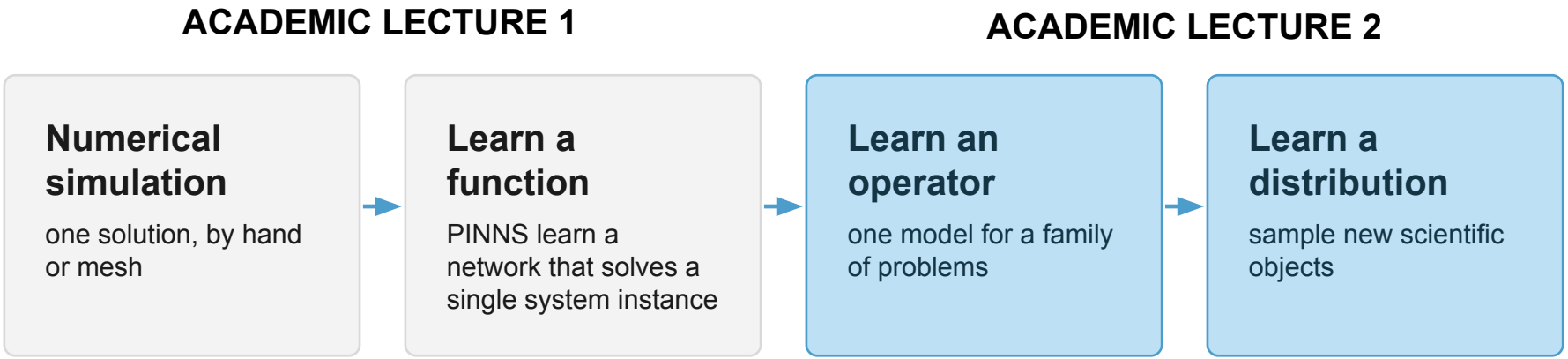
Briefly continue training the pretrained weights on a small, task-specific dataset. The model adapts its general knowledge to the specifics.

Example: Walrus (2025)
A single transformer pre-trained across nineteen physical domains, from astrophysics, geoscience, plasma physics, acoustics, classical fluids, that can then be fine-tuned to new PDE systems.

PART 4

Summary

Summary



Each step makes scientific computation more general and more scalable!